SOCCERIQ-FUNCTION: OFFLINE Q-FUNCTION EVALUATION FOR ESTIMATING SOCCER ACTIONS REWARDS

ETHAN SENATORE [ETHSEN@SEAS], IGNACIO BOERO [IBOERO@SEAS],

ABSTRACT. This paper presents a framework for evaluating soccer actions through offline Q-function estimation, enabling the assessment of long-term action effectiveness without policy optimization. Using spatial state encodings and hybrid action representations, we train neural networks to predict expected returns from historical match data. Our approach employs both Monte Carlo and temporal difference (TD(0)) return estimators on StatsBomb tracking data, with TD(0) achieving superior performance (validation loss 0.12 vs 0.81 for MC) through Bellman equation consistency. This work demonstrates the feasibility of deep Q-approximation for soccer analytics while highlighting critical challenges in state representation and generalization for complex game scenarios.

1. INTRODUCTION

Modeling the game of soccer poses a unique challenge due to its fluidity, strategic depth, and the multitude of factors that influence every play. What may appear to be a negligible action in isolation could, in fact, be instrumental in creating a goal-scoring opportunity several steps later. This difficulty in quantifying long-term value makes it essential to develop robust methods for evaluating actions not just by their immediate outcomes, but by their contribution to future success, such as scoring a goal. Understanding which actions are more likely to lead to goals over time is crucial not only for tactical analysis but also for player analysis. In this day and age, the world's top teams are in a constant battle of being able to more accurately analyze a player's ability and decision making. Possessing even the slightest advantage in analytical ability over a rival could result both, better performance on the field and more lucrative transfer deals being made off the field. Barcelona FC, arguably the world's golden standard as a soccer institution, is one of the many clubs that heavily invests in their analytics team and even sponsors academic research within the field.

One of the major difficulties in applying reinforcement learning to soccer lies in defining appropriate representations for states and actions. Soccer is a continuous, multi-agent environment, so capturing the nuances of each situation is non-trivial. In this project, we approach this challenge using offline Q-Function evaluation, a method from reinforcement learning that enables us to estimate the expected return-i.e., the future reward of actions taken from a given policy, without interacting with the environment. Our goal is not to discover a new or optimal policy, but rather to evaluate the Q-function of the behavior policy from which the data was collected. This enables a practical and scalable way to assess the long-term effectiveness of soccer actions based on historical game data. To approximate the Q-function, we leverage deep learning, specifically a neural network trained to predict future expected return given a state-action pair. In our approach, we represent the state as a spatial image encoding the positions of players and the ball, allowing us to leverage convolutional neural networks (CNNs) for spatial understanding. Actions are represented through a combination of a discrete one-hot encoding-indicating the type of action such as pass, shoot, or dribble-and a continuous component, such as the target position of a pass. These action features are processed by a fully connected neural network (FCNN). The outputs of the CNN and the FCNN form learned embeddings for state and action, which are then input to the Q-network. This structure allows the model to learn rich, context-aware representations that can effectively estimate the expected return of various soccer actions.

2. BACKGROUND AND RELATED WORK

Within the last thirty years, research into soccer has grown substantially due to its continuous nature as opposed to more discrete games. Many ask the question, "What metric does the best job in objectively quantifying a player's ability?" In [1], one of the most notable works in the field, the authors coin xG (expected goal). xG quantifies the probability of a shot being scored based on where it is taken. This metric allows for the careful analysis of a striker's performance, i.e. an underperforming striker typically has a higher xG than actual goals scored and vice versa. xThreat (expected threat) leverages Markov Decision Processes to assign a threat value to each cell of a discretized version of the pitch [2] allowing for analysis of where teams are most threatening. Another use of MDPs is in [3], where the authors analyze the decision making of players around the penalty area. They specifically concentrate their efforts on figuring out whether players should or should not shoot based on their position and the lost opportunity when a decision is made over the other. Similarly, the VAEP (Valuing Actions by Estimating Probabilities) [4] framework evaluates player actions by predicting the probability of scoring from a given state and then computing the difference in value between consecutive states to approximate the value of an action.

Other researchers have used more complicated deep learning architectures to capture more of the nuances of the game. Fernández, Llana, and Peralta ([5] , [6] , [7] , [8]) spatially analyze the game and study the space that the players do and *do not* occupy. SoccerMap [6] proposes a deep learning architecture that uses state representations

similar to ours: spatial images encoding player and ball positions, but focuses on predicting the probability distribution over pass targets across the field rather than estimating long-term value. Unlike our approach, they model state value rather than action value directly and rely on tree-based models, whereas we directly approximate the Q-function using deep neural networks. Another related line of work is the Expected Possession Value (EPV) framework [5], which estimates the value of states by decoupling the action selection and valuation steps. This modular approach contrasts with our more unified model that learns action values directly in an end-to-end fashion. The most closely related effort is presented in [9], where a deep reinforcement learning framework is employed to optimize player decisions in soccer. However, their goal is to discover optimal policies, whereas our focus is purely on evaluating the Q-values of a fixed behavior policy using offline data. Our work is also grounded in the principles of classical reinforcement learning as outlined in [10], particularly the on-policy approximation of action values discussed in Chapter 9. This provides the theoretical foundation for our method, which treats Q-function estimation as a supervised learning problem over collected trajectories, emphasizing accurate function approximation without policy improvement.

3. Approach

We frame the offline policy evaluation problem in soccer as the task of approximating the action-value function $Q^\pi(s, a)$ using supervised learning. This involves training a neural network to estimate the expected return of actions based on historical match data. Our method comprises three main components: (1) approximation of the Q-function using various return estimators, (2) structured representations of game states and actions, and (3) trajectory segmentation and shaped reward design tailored to soccer dynamics.

3.1. Offline Q-Function Approximation.

To estimate the action-value function $Q^\pi(s, a)$, which represents the expected return of taking action a in state s under policy π, we treat the problem as a supervised regression task. Instead of storing values in a table, we approximate $Q^\pi(s, a)$ with a neural network $\hat{Q}(s, a; \mathbf{w})$, where $\mathbf{w} \in \mathbb{R}^n$ are learnable parameters. The network is trained on triplets (s, a, target), minimizing the squared error between its output and a target return:

$$\mathcal{L}(\mathbf{w}) = \Big(\hat{Q}(s, a; \mathbf{w}) - \text{target}\Big)^2$$

We use stochastic gradient descent or its variants (e.g., Adam) to optimize this loss over a dataset of offline trajectories generated by a behavior policy π.

Choosing the target return is a key design choice. We explore two standard estimators. The first is the **Monte Carlo (MC)** return:

$$\text{target}_{\text{MC}} = G_t = \sum_{k=0}^{T} \gamma^k R_{t+k+1}$$

MC targets are unbiased but often exhibit high variance in long episodes. The second is the **TD(0)** target, which bootstraps from the next state:

$$\text{target}_{\text{TD(0)}} = R_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}; \mathbf{w})$$

This method reduces variance but introduces bias, especially early in training.

3.2. State and Action Space.

To represent the state space, we adapt the spatial encoding methodology from SoccerMap[6], which generates image-like representations of player formations using tracking data. Due to the absence of velocity information in our dataset, we adopt a simplified version. Each state is encoded as a multi-channel tensor over a discretized soccer field of size 104×68, with each cell representing approximately one square meter. The state includes four channels:

- Two sparse matrices indicating player positions for the attacking and defending teams.
- Two dense matrices encoding the distance and angle from each grid cell to the ball location.

The implementation of the network closely follows the work done in [11] as we leverage the same data. Actions consist of both discrete and continuous components. The discrete component is a one-hot vector over three action types: *pass*, *dribble* and *shot*. The continuous component encodes spatial details. For passes and dribbles, we include coordinates $(x_{\text{start}}, y_{\text{start}}, x_{\text{end}}, y_{\text{end}})$. For shots, only the starting location is relevant; the remaining values are set to zero. = To process the inputs, we use two separate encoders. The **state tensor** is passed through a Convolutional Neural Network (CNN) to extract spatial features. The **action vector**, including both one-hot and continuous features, is processed through a Fully Connected Neural Network (FCNN) to yield a dense action embedding. These embeddings are concatenated and passed to the Q-network, which predicts the expected return $Q^\pi(s, a)$ for each state-action pair.

3.3. **Trajectories and Rewards.** We define a trajectory as a sequence of actions taken by a team during a single uninterrupted possession. A trajectory starts when a team gains control of the ball and ends when possession is lost-through an interception, a missed shot, or a goal. This segmentation enables the decomposition of continuous play into discrete, learnable units. Using event logs and tracking data, we extract such trajectories across the dataset.

Reward design is particularly challenging in soccer due to the rarity of goals. A naive +1 reward for goals yields sparse and inefficient learning signals. To address this, we use a shaped reward function that provides graded feedback based on the outcome of each trajectory. For instance, a goal yields the highest reward, followed by a shot on target. A shot off target receives a smaller positive value, while losing the ball in the opponent's half yields zero. Possession loss in the defending half results in a negative reward. This shaped scheme encourages the model to distinguish high-impact actions even when goals are infrequent. The specific numeric values where selected empirically and are +100 for a scored goal, +20 for a on target shot, +10 for a off target shot, -10 for a ball lost in opponents field half and -20 for ball lost in own field half.

4. Numerical Experiments

Experiments were trained using a GTX4090. The code used for the implementation can be found in our github repo.[1]

4.1. **Dataset.** We use 360 tracking and event data from the StatsBomb 360 open dataset. This dataset provided us with 292 matches containing over 1,000,000 events. We converted each event into the Soccer Player Action Description Language (SPADL) [4] format, which allowed us to easily vectorize the events in the dataset. There are 27 total different actions an event can hold, but we only cared for pass, dribble, cross, and shoot. The 360 tracking data is acquired at each event through a computer vision approach that tracks players through the broadcast feed of soccer games. Although this approach is great for open source researchers, it lacks the resolution that traditional tracking data has. Often times, events are missing players and key details regarding the game state. This differs from what other works mentioned in Section 2 use. Typically, higher resolution tracking data that records the positions of the player and the ball at 10 Hz is coupled with event data. Unfortunately, getting access to such data is fairly difficult.

4.2. **Architecture.**
State encoder.

$$\phi_s : \mathbb{R}^{4\times 104\times 68} \xrightarrow{\text{Conv}(4\to 32)} \text{ReLU} \to \text{MaxPool}(2) \xrightarrow{\text{Conv}(32\to 64)} \text{ReLU} \to \text{MaxPool}(2) \xrightarrow{\text{Conv}(64\to 128)} \text{ReLU} \to \text{Flatten}$$
$$\xrightarrow{\text{Linear}(128\times 3\times 2\to 256)} \text{ReLU} \longrightarrow \mathbb{R}^{256}.$$

Action encoder.

$$\phi_a : \mathbb{R}^{7} \xrightarrow{\text{Linear}(7\to 128)} \text{ReLU} \xrightarrow{\text{Linear}(128\to 128)} \text{ReLU} \longrightarrow \mathbb{R}^{128}.$$

Q-network. The concatenated embedding $[\phi_s(s_t), \phi_a(a_t)] \in \mathbb{R}^{256+128}$ is processed by a stack of L fully-connected layers (each followed by `ReLU`); a final linear read-out produces the scalar estimate $\hat{Q}(s_t, a_t) \in \mathbb{R}$.

4.3. **Implementation Details.**

4.3.1. *Train/Val split.* To avoid different distributions between the training and validation sets, we shuffled the events of all the trajectories in all the games and divided 80% for training and 20% for validation. As the dataset was too large to fit into memory, both validation and training sets were divided into shards, which were accessed sequentially by an iterative data loader.

4.3.2. *Normalization.* For every input feature the empirical mean and standard deviation was calculated over the complete training set. Then, before feeding the states and actions to the encoders, its values are normalised. Similarly, the target values for the return are normalized.

4.3.3. *Regularization.* To avoid overfitting to the training data the training data we used the following regularization approaches:

- **Weight decay**: ℓ_2 penalty to the weights with a coefficient in $[10^{-6}, 10^{-3}]$.
- **Dropout**: applied in both encoders and Q-head with a shared rate $p_{\text{drop}} \in [0, 0.20]$.
- **Gradient clipping**: ℓ_2-norm capped at 1.0.
- **Target network**: For $TD(0)$ approach we kept a frozen copy of the Q-function network for the return estimation. The network was updated every 2000 iterations.
- **Early stopping**: training stops after three epochs with no validation improvement.

Hyper-parameter	Search space
Learning rate η	$\log U\big(5 \times 10^{-5},\, 10^{-3}\big)$
Weight decay λ	$\log U\big(10^{-6},\, 10^{-3}\big)$
Batch size B	$\{32,\, 64,\, 128\}$
Number of hidden layers L	$\{2,\, 3,\, 4,\, 5\}$
Hidden dimension h	$\{64,\, 128,\, 256\}$
Dropout rate p_{drop}	$\{0.00,\, 0.05,\, 0.10,\, 0.15,\, 0.20\}$

TABLE 1. Hyper-parameter search space explored during optimisation.

Hyperparameter	TD(0)	Monte Carlo
Learning rate (η)	3.2×10^{-4}	6.7×10^{-4}
Weight decay (λ)	2.5×10^{-5}	4.1×10^{-6}
Batch size (B)	64	128
Number of hidden layers (L)	3	4
Hidden dimension (h)	128	256
Dropout rate (p_{drop})	0.10	0.05
Training loss	**0.08**	0.56
Validation loss	**0.12**	0.81

TABLE 2. Best hyperparameter configurations and performance.

4.3.4. *Grid search.* To find the best set of hyperparameters we did a gridsearch for both the TD(0) and montecarlo method over the parameters shown in table 1. The search was performed by Optuna with 64 trials for each method. The hyperparameter configuration for both models can be found in Table 2. The TD(0) model achieved a final training loss of 0.08 and a validation loss of 0.12. The Monte Carlo model achieved a training loss of 0.56 and a validation loss of 0.81. In this case the generalization difference was notably larger. We note that the loss values are not directly comparable between methods. In the TD(0) setup, the loss reflects the temporal-difference error, specifically the discrepancy $Q(s,a) - [r + Q(s',a')]$, which measures the inconsistency with the Bellman equation across consecutive state-action pairs. In contrast, the Monte Carlo setup defines the loss as the error between the predicted $Q(s,a)$ and the full return estimate, making the nature and scale of the losses fundamentally different. The main issue we faced was that the models were not able to generalize well, which we attribute mainly due to the poor quality of the data. In many data samples, the players location was not coherent with the ball start and end location. A more significant analysis than watching the loss, is to see the actual Q-function value for different actions that can be taken for a given state.

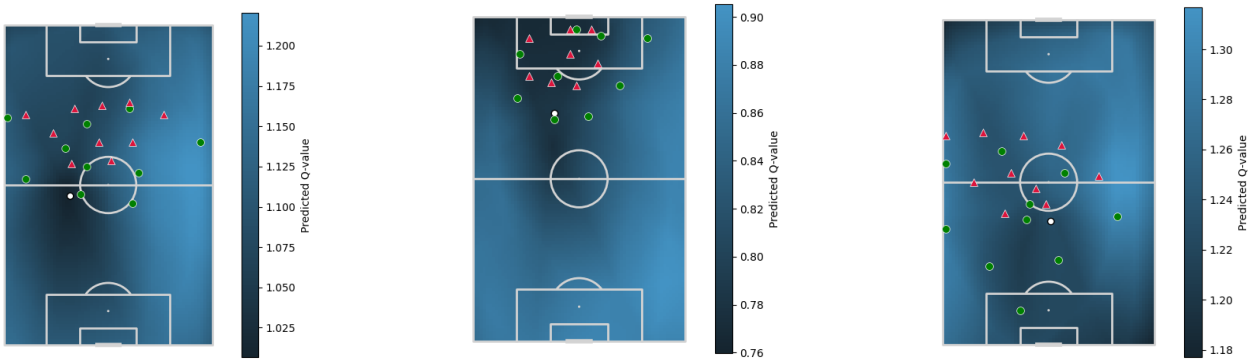

FIGURE 1. Three examples of a game state and their respective Q-value heatmaps of the reward to pass to a cell on the field. (Far Left) Predicted Q for SHOT: -1.129. (Middle) Predicted Q for SHOT: 0.848. (Far Right) Predicted Q for SHOT: -1.102. Green dots are teammates, red triangles are opponents, and the white dot is the ball.

In Figure 1, we show three examples of our model outputs after processing a given game state. For each example, a reward heatmap is generated in which our model determines the potential reward the actor would receive, given that they pass into the specified area. On the far left, the game state is in a classical possession state where-by the defending team lies in a mid-block in their own half. Traditionally, the team in possession would seek to maintain possession of the ball while trying to exploit the space left open by the opposing team. The model does a fine job in highlighting the open space on the right-hand side of the pitch and does well in identifying the less rewarding path to goal through the middle. In the middle, the game state is in a classical *last phase* state where-by the team in possession is just outside the penalty area carefully crafting an opportunity to strike. Much of the modern game is focalized on this exact phase, and has become one of the central focuses of research. The model does well in analyzing the clogged space within the box

[1] https://github.com/IgnacioBoero/Soccer

and determines the higher reward area to be on the outside of the penalty area. On the right-hand side, the game state is in the *build-up* phase where-by the in possession team looks to build an attack from the back. Once again, the model is able to highlight the open areas the side of pitch. For all three examples, the model assigns high rewards if the actor plays backward which can be attributed to two potential reasons. First, our model may prioritize safety and possession of the ball, thus playing backwards seems like the best possible way to prioritize possession netting a higher reward. This could indicate that the model has learned a meaningful pattern, but the second and more likely reason, is that this is a byproduct of model bias or data artifacts. Another interesting detail to note is that, when the game state is in the *last phase*, rewards seem to decrease. Making the correct decision in and around the penalty area should, in theory, lead to the highest possible reward. However, our model appears to assign a higher expected reward to passes made near the middle of the pitch. At this stage, we lack sufficient evidence to determine whether this attribution is due to genuine strategic insight or an accidental outcome. Further analysis is needed to better understand the reasoning behind this pattern. During testing, it was difficult to find meaningful examples due to the quality of the data and the inability for our model to handle complex states. We found that the more context given to the model, the better it performed. If the sample was incoherent or lacked most of the players on the pitch, the model simply outputted a meaningless heatmap.

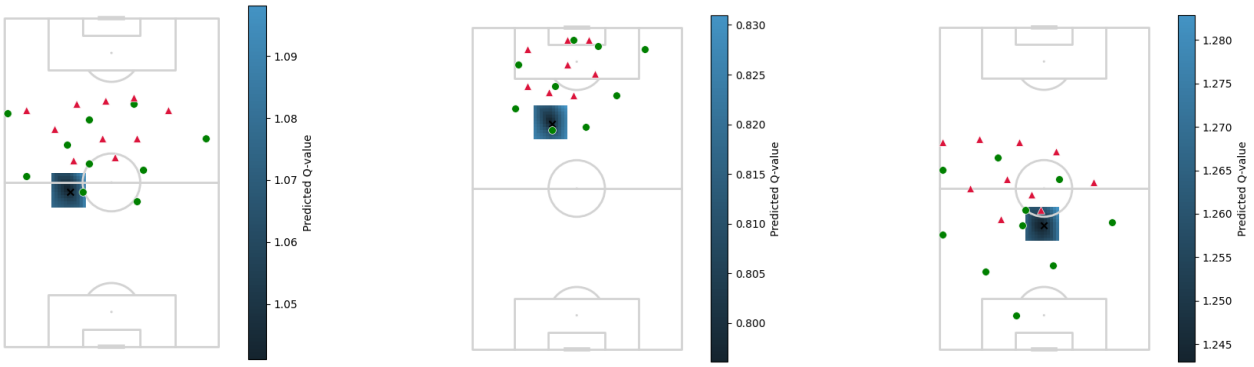

FIGURE 2. Three examples of a game state and their respective Q-value heatmaps of the reward to dribble within a 5m x 5m surrounding area Green dots are teammates, red triangles are opponents, and the X mark is the ball.

Figure 2 shows a snapshot of a game state along with a 5m × 5m surrounding area heatmap illustrating the model's evaluation of reward values for dribbling actions, indicating where it believes the most advantageous movements lie. This was by far the weakest aspect of the model as the results lack any sort of clarity in advising or advising against certain dribbling actions. We deduced that not enough relevant data was used to train the model, and most of the trajectories possessing high rewards came from pass-to-shoot sequences.

5. DISCUSSION

The results demonstrate that direct Q-function approximation via deep networks can capture some meaningful spatial patterns in soccer, but also highlights key challenges in data quality, representation and evaluation. Compared to existing value-based approaches (e.g., VAEP or EPV), our model offers an end-to-end estimate of action-values, yet struggles with generalization and nuanced decision-making. Our TD(0) estimator achieved low temporal-difference error on held-out data, indicating successful fitting of the Bellman equation, whereas the Monte Carlo baseline suffered high variance and poorer generalization . The generated pass-reward heatmaps sometimes highlight open space, mirroring SoccerMap's spatial awareness , but also occasionally prioritize backward passes in high-reward scenarios-suggesting the model learned a "safe play" bias rather than true offensive value.

Assuming we had more time and access to a higher resolution dataset, we would have been interested in further refining our approach. First and foremost, we wished we could have conducted a more thorough analysis of our model and determine whether our Q-learning method was able to capture more of the nuances of the game. Additionally, we suspect that our approach was very sensitive to the quality of the data, and unfortunately, we did not allot enough time to clean the data. Secondly, we completely ignored the defensive action space that the out-of-possession team has access to. We suspect that conducting an analysis in this adjacent, yet quite different, domain would have revealed interesting patterns.

REFERENCES

[1] Richard Pollard and Charles Reep. Measuring the effectiveness of playing strategies at soccer. *Journal of the Royal Statistical Society Series D: The Statistician*, 46(4):541–550, 1997.

[2] Karun Singh. Introducing expected threat (xt). `https://karun.in/blog/expected-threat.html`. Accessed: 2025-05-09.

[3] Maaike Van Roy, Pieter Robberechts, Wen-Chi Yang, Luc De Raedt, and Jesse Davis. Leaving goals on the pitch: Evaluating decision making in soccer. *arXiv preprint arXiv:2104.03252*, 2021.

[4] Tom Decroos, Lotte Bransen, Jan Van Haaren, and Jesse Davis. Actions speak louder than goals: Valuing player actions in soccer. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 1851–1861, 2019.

[5] Javier Fernández, Luke Bornn, and Dan Cervone. Decomposing the immeasurable sport: A deep learning expected possession value framework for soccer. In *13th MIT Sloan Sports Analytics Conference*, volume 2, 2019.

[6] Javier Fernández and Luke Bornn. Soccermap: A deep learning architecture for visually-interpretable analysis in soccer. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 491–506. Springer, 2020.

[7] Sergio Llana, Pau Madrero, Javier Fernández, and F Barcelona. The right place at the right time: Advanced off-ball metrics for exploiting an opponent's spatial weaknesses in soccer. In *Proceedings of the 14th MIT Sloan Sports Analytics Conference*, 2020.

[8] F Peralta Alguacil, J Fernandez, P Piñones Arce, and D Sumpter. Seeing in to the future: using self-propelled particle models to aid player decision-making in soccer. In *Proceedings of the 14th MIT sloan sports analytics conference*, 2020.

[9] Pegah Rahimian, Jan Van Haaren, Togzhan Abzhanova, and Laszlo Toka. Beyond action valuation: A deep reinforcement learning framework for optimizing player decisions in soccer. In *16th MIT sloan sports analytics conference*, 2022.

[10] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[11] Pieter Robberechts, Maaike Van Roy, and Jesse Davis. Un-xpass: Measuring soccer player's creativity. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, page 4768–4777, New York, NY, USA, 2023. Association for Computing Machinery.